

Early Iterations of Instruction Cache Design

Some contents in this file were last updated 29 May 2026. Other documents may contain:

- *Newer CPI and timing data*
- *More in-depth documentation for the final cache design*

However this file is still useful to serve as a document of how the cache iterative design process was carried about.

This document contains rough notes of the different iterations of the instruction cache, and brief justifications for why certain designs were tried.

Please see the icache source files: `picosoc/icache.v` for the final designs and `picosoc/icache_design_iterations.v` for these earlier iterations. There may be more detailed comments in the code files.

Preliminary Results as of 29/05/2026

Preliminary Results	
Integral Benchmark	CPI
Baseline (No Cache)	9.9374
128x16 Cache	5.9998
64x16 with lookahead	5.0736

Bubble Sort Benchmark	CPI
Baseline (No Cache)	10.991
128x16 Cache	5.5699
64x16 with lookahead	4.5159

Cache Iterations

All the iterations of the instruction cache go in between the cpu's memory interface and the memory interface in `picosoc.v` (where the cache is instantiated), preserving the shape of the interface on both sides. There are many versions of the cache, with description/rationale.

"Good" iterations (ones where changes were kept and built upon for further generations) are marked with (*)

The main one to be used for the final processor is `icache_multiword_lookahead` or `icache_multiword_lookahead_2way`.

icache_zerocycle

- This is a very basic cache with no state machine, and entirely combinational (direct pass-through of memory (except when valid signal is intercepted for a cache hit, and returned ifetch data is copied into the cache). While it was useful as a development step, it has little practical use, since putting it on BRAM would require 1-cycle read but putting it on LCs means maximum cache size is too small to be practical.
- One word per line, maximum 8-16 lines on LCs only.

icache (*)

- Basic cache, now with a 1-cycle lookup delay driven by a simple 3-state FSM. Still combinational pass through except for ifetch instructions. If it's an ifetch then we wait a cycle for the BRAM access, then process hit/miss.
- One word per line, maximum 128 lines (MUX/line select logic gets expensive fast). This is a decent amount - basic polynomial integral benchmark is around 27 lines.

icache_first_miss_bypass

- Same as icache, with a small change: cache lines are only updated upon the second miss (to make the cache "sticky"). This was intended to reduce thrashing for when the cache is too small for the hot-loop (or hot loop has some instructions that are very hot and other instructions that are called once in a while) (e.g. if the hotloop is 32 instructions and the cache is only 16 lines, we would prefer some 16 lines still stay there, instead of constant misses).
- But upon testing the benefit was not high enough to justify the larger LC cost.

icache_random_bypass

- Same idea as first_miss_bypass but we only update cache instructions with a certain probability (e.g. 25% or 50%) every miss. This is intended so that occasional instructions are less likely to be in the cache for a long time and hot instructions can stay in cache more. Also was intended so performance deterioration isn't as sudden.
- Upon testing this was rather ineffective.

icache_multiword (*)

- Based on icache, but a significant architectural change to allow multiple words per line
- This is a valuable change because currently we're limited by maximum number of lines (~128) due to muxing logic, storing tag/valid, etc, but the BRAM still had space (128x1 used 8/30).
- On miss, fill the whole line (multiple words) - so there's a trade-off between cache size (words per line) and # of potentially unnecessary reads
- Has a 4-state FSM for IDLE, LOOKUP (check if hit or miss), FILL, RESP, and an internal counter to keep track of which line is being filled
- Approx largest size that fits is 128x16 (2,048 instructions) - that's massive.

icache_multiword_first_miss_bypass

- Same rationale as icache_first_miss_bypass applied to icache_multiword
- This was ineffective. Costs a lot of extra logic, and when there are multiple words per line and it misses once, it was probably going to miss again anyway.

icache_multiword_lookahead (*)

- Same as icache_multiword, but uses the CPU lookahead interface to allow the cache to find the cached data early, so now when the actual ifetch comes we can respond to hits with zero-cycle delay again! Saves close to 1CPI on benchmarks compared to regular multiword cache (but may need smaller cache to fit in additional logic).
- Lookahead also removes the need for the 1-cycle delay LOOKUP state so we only need 3-state FSM
- Largest working size 128x16

icache_multiword_lookahead_2way (*)

- Functionally the same as icache_multiword_lookahead, but adds associativity (2-ways)
- Largest working size 64 sets x 2 ways x 16 words per line (2048 instructions)
- Larger area and very slightly worse timing than non-associative version, but should have reduced conflict misses